

O desenvolvimento de software tem passado por diversas transformações ao longo dos anos, principalmente com o surgimento de metodologias ágeis. Essas metodologias propõem ciclos de desenvolvimento mais curtos, com entregas frequentes de novas versões do sistema. A ideia é que os times consigam receber feedback rapidamente e, se necessário, ajustar o rumo do projeto. No entanto, mesmo com práticas ágeis sendo adotadas no desenvolvimento, muitas empresas ainda enfrentam dificuldades quando chega o momento de colocar novas versões do sistema em produção. Esse problema está relacionado, em grande parte, ao tipo de arquitetura utilizada pelas aplicações.

Tradicionalmente, muitos sistemas são desenvolvidos seguindo uma arquitetura monolítica. Nesse modelo, mesmo que o código seja organizado em diversos módulos durante o desenvolvimento, todos eles são executados como um único processo quando o sistema está rodando. Isso significa que todos os componentes compartilham o mesmo espaço de memória e estão fortemente conectados entre si. Na prática, isso pode gerar problemas, pois uma alteração feita em um módulo pode acabar afetando outros módulos de maneira inesperada.

Esse tipo de situação faz com que as empresas adotem processos muito cautelosos antes de liberar novas versões do sistema. Normalmente são realizados diversos testes para garantir que nenhuma funcionalidade foi comprometida. Em alguns casos, até mesmo testes manuais são feitos, simulando o uso real do sistema por um usuário. Embora essas práticas ajudem a evitar falhas, elas também tornam o processo de implantação mais lento e burocrático, o que vai contra a proposta de agilidade defendida pelos métodos modernos de desenvolvimento.

Nesse contexto surge a arquitetura baseada em microsserviços. A ideia principal dessa abordagem é dividir o sistema em vários serviços menores e independentes, cada um responsável por uma parte específica da aplicação. Diferente do modelo monolítico, esses serviços são executados em processos separados, o que reduz o acoplamento entre as partes do sistema. Dessa forma, mudanças feitas em um serviço têm menos chance de afetar diretamente outros serviços.

Segundo o artigo de Martin Fowler sobre o tema, microsserviços podem ser entendidos como uma forma de desenvolver aplicações compostas por pequenos serviços autônomos que se comunicam entre si por meio de mecanismos simples, geralmente através de APIs. Cada serviço pode ser desenvolvido, implantado e atualizado de forma independente, o que facilita muito a evolução do sistema ao longo do tempo.

Uma das principais vantagens desse modelo é justamente a autonomia das equipes de desenvolvimento. Como cada serviço é responsável por uma funcionalidade

específica, diferentes equipes podem trabalhar em partes distintas do sistema sem interferir diretamente no trabalho umas das outras. Isso torna o desenvolvimento mais organizado e permite que novas funcionalidades sejam implementadas e disponibilizadas com mais rapidez.

Outro ponto importante é a questão da escalabilidade. Em sistemas monolíticos, quando ocorre um aumento no número de usuários ou no volume de requisições, normalmente é necessário replicar toda a aplicação em novos servidores. Isso significa que todos os componentes do sistema são duplicados, mesmo aqueles que não estão enfrentando problemas de desempenho. Já em arquiteturas baseadas em microsserviços, é possível escalar apenas os serviços que realmente precisam de mais recursos. Por exemplo, se o serviço responsável pela autenticação de usuários estiver sobrecarregado, apenas ele pode ser replicado em novas instâncias.

Além disso, microsserviços permitem maior flexibilidade no uso de tecnologias. Como cada serviço é independente, diferentes linguagens de programação, frameworks e tipos de banco de dados podem ser utilizados dentro do mesmo sistema. Um serviço pode ser desenvolvido em Java utilizando um banco de dados relacional, enquanto outro pode utilizar Python com um banco de dados NoSQL. Essa liberdade possibilita que cada equipe escolha as ferramentas mais adequadas para resolver o problema que está enfrentando.

Outro benefício importante está relacionado à tolerância a falhas. Em sistemas monolíticos, uma falha em um componente crítico pode acabar derrubando todo o sistema. Em uma arquitetura de microsserviços, porém, as falhas podem ser isoladas. Caso um serviço específico fique indisponível, outros serviços podem continuar funcionando normalmente. Por exemplo, em um site de comércio eletrônico, o sistema pode continuar permitindo compras mesmo que o serviço responsável por recomendações de produtos esteja temporariamente fora do ar.

Os textos também destacam que o crescimento dessa arquitetura está diretamente ligado ao avanço da computação em nuvem. Plataformas de nuvem permitem criar e gerenciar rapidamente máquinas virtuais e serviços de infraestrutura, o que facilita muito a implantação e a escalabilidade de aplicações baseadas em microsserviços. Dessa forma, as empresas conseguem aumentar ou reduzir recursos conforme a demanda do sistema.

Outro conceito interessante mencionado é a Lei de Conway, formulada por Melvin Conway em 1968. Essa lei afirma que a estrutura dos sistemas de software tende a refletir a estrutura organizacional das empresas que os desenvolvem. Em outras palavras, sistemas desenvolvidos por organizações com várias equipes independentes acabam sendo divididos em vários componentes independentes. Isso ajuda a explicar

por que a arquitetura de microsserviços é muito comum em grandes empresas de tecnologia, que possuem diversas equipes trabalhando simultaneamente em diferentes partes de um mesmo sistema.

De forma geral, os textos analisados mostram que a arquitetura de microsserviços surgiu como uma alternativa às limitações das arquiteturas monolíticas. Ao dividir o sistema em serviços menores e independentes, essa abordagem permite maior flexibilidade, escalabilidade e autonomia para as equipes de desenvolvimento. Com isso, torna-se mais fácil evoluir sistemas complexos e acompanhar as necessidades de um ambiente tecnológico cada vez mais dinâmico.